Portfolio Manager - Team 7

By: Antonina, Abraham, Annie

TABLE OF CONTENTS

INTRODUCTION

01

INTRODUCTION

Antonina's Background

- Graduated from John Abbott
- Computer Science DCS
- Mostly did C#, Python, TypeScript
- Currently in Memorial University B.Tech in Engineering & Applied Sciences (part-time CAP)

Annie's Background

- Graduated from NYU
- BA in Data Science
- Mostly did Python, Java, ML, Data Modeling

Abraham's Background

- Graduate from Indiana Tech
- Software engineering
- Python,C#,SQL

Project Overview

02

Project Intro

What We Were Asked to Build

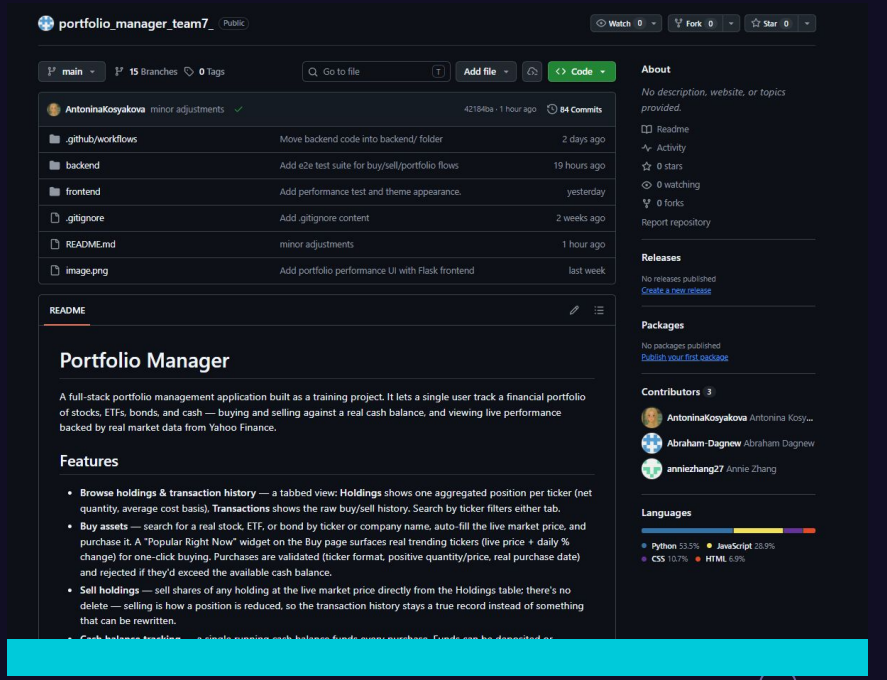
- A Portfolio Manager: track stocks, ETFs, bonds, and cash
- REST API to save and retrieve portfolio data
- Frontend priorities: browse → view performance graphically → add → remove
- Single user, no authentication

What We've Been Learning

- Sprint 1 - Linux, Python fundamentals, SQL/databases, AI code assistants
- Sprint 2 - Version control with Git (branching, PRs)
- Sprint 3 - Networking fundamentals, public cloud fundamentals, **Flask and REST**
- Sprint 4 - HTML, CSS, JavaScript

How Much Time We've Had

- *3 weeks*
- Three phases: core API + database → frontend → open-ended enhancements



Team approach and Workflow

03

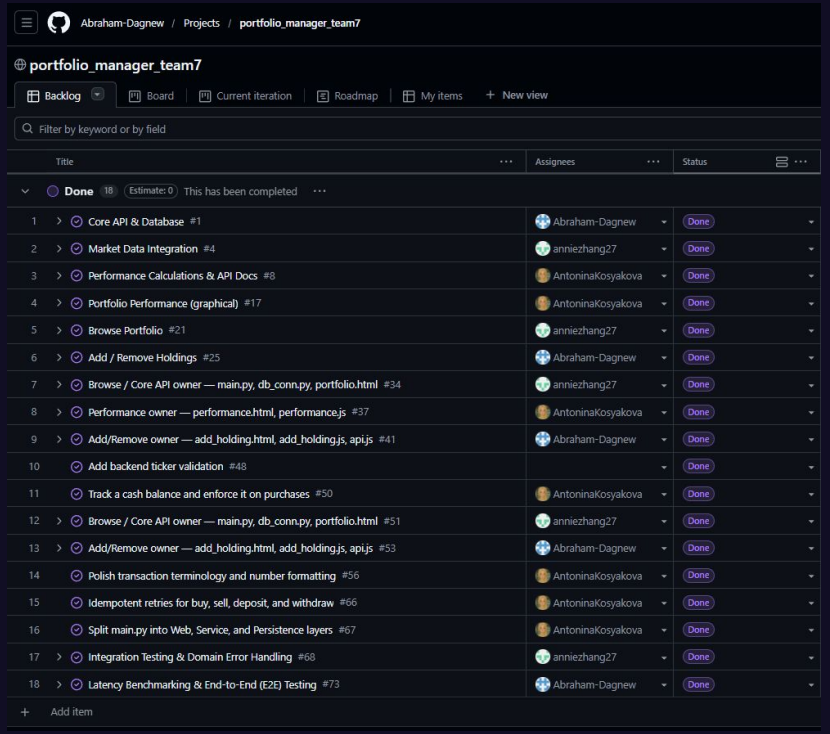
Team approach and workflow

Team split

- Started 3-way by feature:
  Core API & DB · Market Data · Performance/Docs
- Shifted to full-stack ownership per feature as frontend work began
- Later phase: cross-cutting work (validation, balance tracking, refactor) picked up as gaps appeared

Tools & Process

- GitHub Projects backlog - 15 completed items
- Git branching + PR review on every feature
- CI (GitHub Actions) - full test suite on every push



Data Model

04

DATA MODEL

MINIMAL START

- First version: just one table (Portfolio) with id, ticker, and quantity

- Deliberately avoided over-designing the schema up to avoid confusion.

- Just show what the user owns. No buy/sell distinction and cash tacking.

GREW FEATURE BY FEATURE

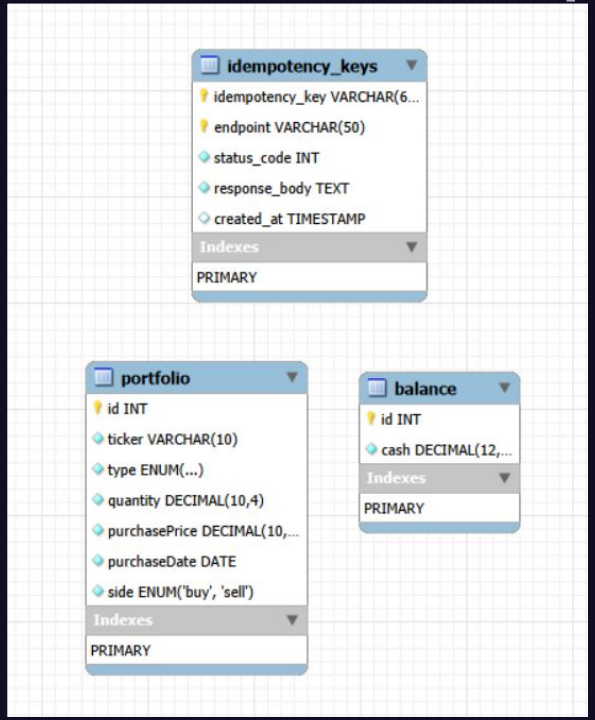
-type (stock / ETF / bond / cash) added once we needed to distinguish asset classes

-purchasePrice, purchaseDate added for performance/gain calculations

-side (buy / sell), the schema evolved from "a list of holdings" into a **transaction ledger**

-balance table, added when we needed a realistic constraint: you can only buy what you can afford

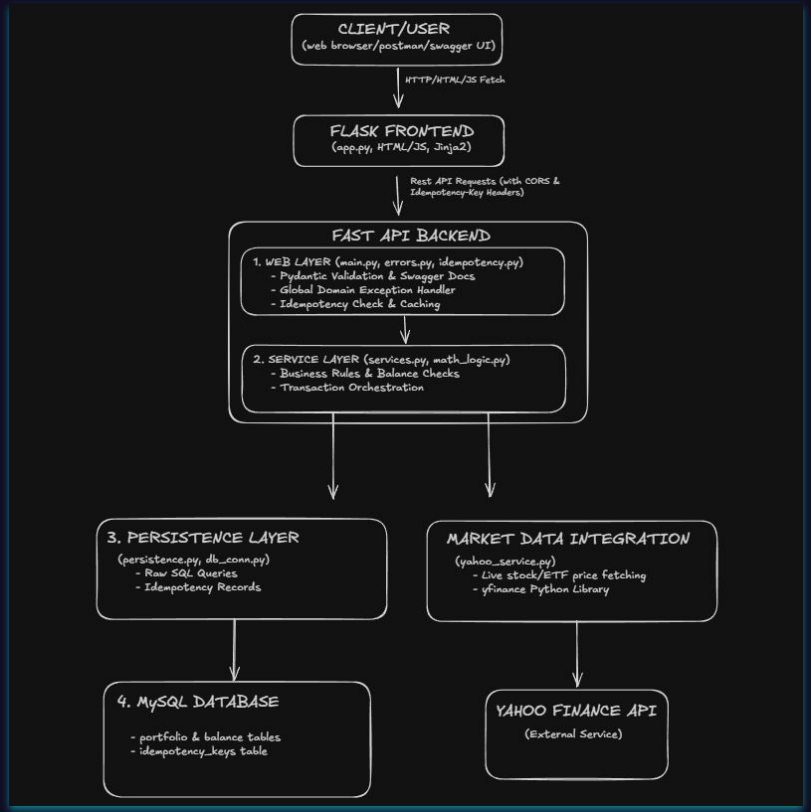
-idempotency_keys table added last, to make retried requests safe



System Architecture

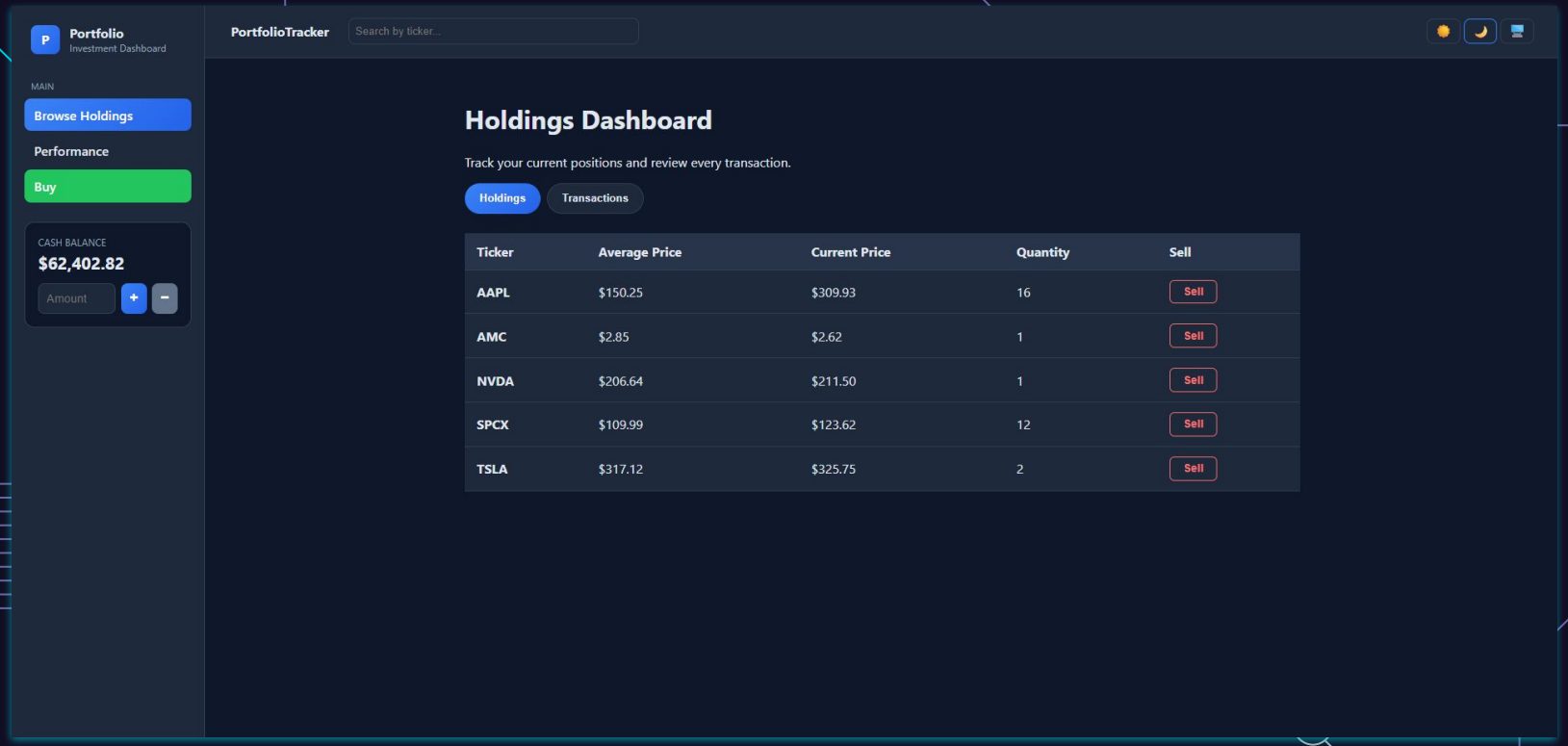
05

System Architecture



Live Demo

06


Portfolio
Investment Dashboard
MAIN
Browse Holdings
Performance
Buy
CASH BALANCE
$62,402.82
Amount
+
−
PortfolioTracker
Search by ticker...
Holdings Dashboard
Track your current positions and review every transaction.
Holdings
Transactions
Ticker
Average Price
Current Price
Quantity
Sell
AAPL
$150.25
$309.93
16
Sell
AMC
$2.85
$2.62
1
Sell
NVDA
$206.64
$211.50
1
Sell
SPCX
$109.99
$123.62
12
Sell
TSLA
$317.12
$325.75
2
Sell

Challenges Faced

07

Technical Challenges & Solutions

Live Market Data Consistency

Handled external network variability and invalid ticker queries with defensive backend error handling.

Single Environment & Lack of Staging

Lacked a dedicated QA/Staging environment, requiring local mock fixtures and database reset hooks to safely execute automated test runs without corrupting persistent data.

Data Integrity & Transactional Idempotency

Engineered a custom Idempotency-Key engine to prevent duplicate trades caused by accidental double-clicks or browser retries

Next Steps

08

What We'd Do Differently & Next Steps

DB Schema Normalization (3NF)

Current: Asset types (stock, etf, bond) are stored directly on every transaction row.

Next Step:

Normalize into 3NF by extracting an assets reference table linked via Foreign Keys to eliminate data redundancy.

Performance & Query Indexing

Current: Portfolio and transaction queries rely on full-table scans or default primary keys.

Next Step:

Add B-Tree indexes (e.g., CREATE INDEX idx_ticker ON portfolio (ticker)) to maintain sub-millisecond query execution as scale grows.

Multi-User Authentication & Scalability

Current: Built as a single-tenant prototype without user account isolation or authentication.

Next Step:

Implement multi-user authentication and migrate MySQL to an auto-scaling managed database (like AWS Aurora) to support concurrent user traffic.

Thank you for listening!

Any questions?